



SEPTIEMBRE DE 2024

PATRONES DE DISEÑO GOF

DISEÑO DE SISTEMAS DE INFORMACIÓN

AYUDANTE MARCOS ESPECHE, SUPERVISADO POR PROF. CLAUDIA POBLETE
UNIVERSIDAD TECNOLÓGICA NACIONAL – FACULTAD REGIONAL MENDOZA

Introducción

Los **patrones** son pares problema-solución, identificados por un nombre, y que se pueden aplicar en contextos nuevos [1]. Resuelven problemas comunes encontrados en el ámbito de la Ingeniería de Software. Existen patrones para el análisis, la arquitectura, el diseño, y distintas etapas del ciclo de vida de desarrollo de software. Como su nombre indica, los patrones de diseño resuelven problemas vinculados al diseño de sistemas.

Un grupo de cuatro ingenieros (Erich Gamma, Richard Helm, Ralph Johnson, y John Vlissides) escribió en 1994 el libro *“Design Patterns: Elements of Reusable Object-Oriented Software”*. El libro presenta 23 patrones con ejemplos escritos en C++ y se los conoce como “Patrones de Diseño GoF” (que viene del inglés *Gang of Four*). Se los agrupa en tres categorías:

- **Creacionales:** Resuelven problemas relacionados con la instanciación de clases, permitiendo una mayor flexibilidad y reutilización.
 - Factory
 - Singleton
 - Abstract Factory
 - Builder
 - Prototype.
- **Estructurales:** Indican cómo relacionar clases para incrementar la escalabilidad y el mantenimiento del código.
 - Adapter
 - Bridge
 - Composite
 - Decorator
 - Facade
 - Flyweight
 - Proxy
- **De comportamiento:** Permiten lidiar con la asignación de responsabilidades vinculada a algoritmos y el envío de mensajes entre objetos.
 - Strategy
 - Chain of Responsibility
 - Interpreter
 - Command
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State
 - Template
 - Visitor

Durante el desarrollo de este apunte, se explicarán algunos de los patrones mencionados, suponiendo requisitos adicionales de dos negocios distintos. Se recomienda tener, previo a esta lectura, sólidos conocimientos sobre los patrones GRASP, UML, orientación a objetos, Java, y el apunte de cátedra “Utilización de Indirección de Persistencia”.

Estrategia

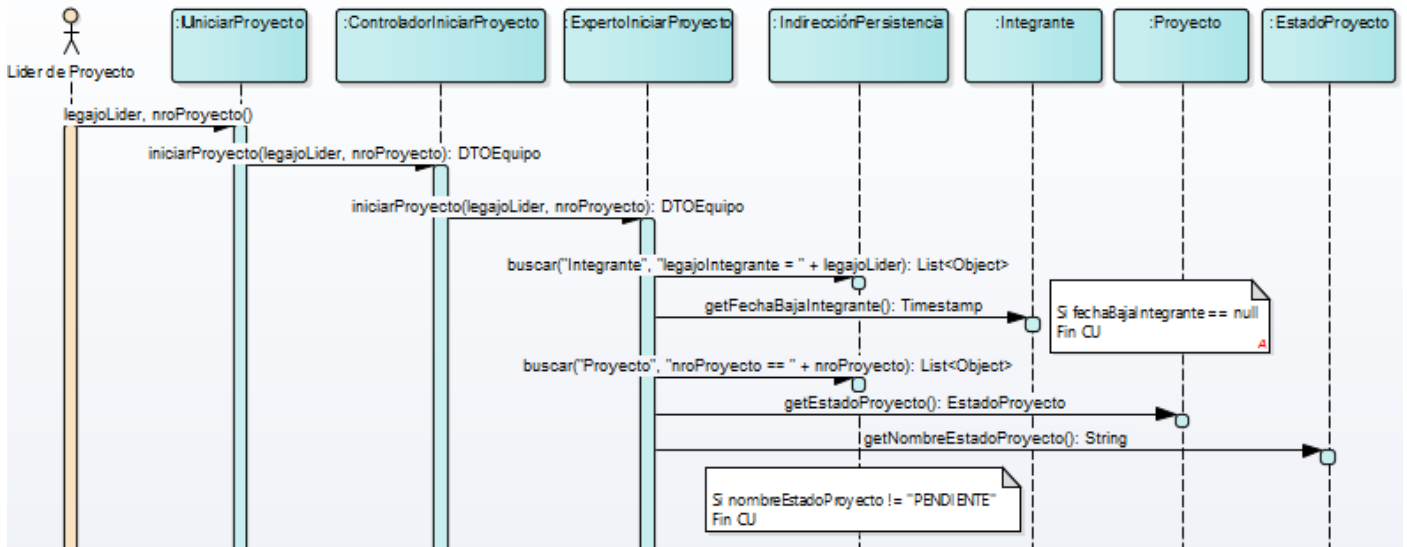
Una empresa se dedica a realizar proyectos de software para distintos clientes. Para ello, dispone de una variedad de equipos a su disposición. Cada uno está conformado por integrantes que pueden desempeñar distintos roles. El equipo es seleccionado automáticamente cuando un líder de proyecto decide tomar un proyecto disponible. Se requiere un mecanismo de selección de equipos flexible y personalizable.

Al registrarse un cliente se especifica su criterio de prioridad para la selección de equipos. Por ejemplo:

- El sistema debe permitir a los clientes cambiar sus criterios de priorización en cualquier momento. Además, se busca que se pueda incorporar nuevos algoritmos de priorización en el futuro.*



Primeramente, se debe validar que el proyecto pueda tomarse y que el lider de proyecto esté registrado en el sistema. Un posible diagrama de secuencia es el siguiente:



A partir de ese punto inicia la obtención del equipo recomendado. Inicialmente, dicha lógica podríamos asignársela a la clase **ExpertoIniciarProyecto**, ya que es **Experto en Información** sobre esa responsabilidad. Sabemos que la forma de obtener dicho equipo varía, ya que según el algoritmo elegido por el cliente se priorizan equipos con características determinadas. Entonces en un principio parece razonable realizar un código como el siguiente:

```
public class ExpertoIniciarProyecto {
    public void iniciarProyecto(int legajoLider, int nroProyecto) throws Exception {
        List<Object> integranteList = IndireccionPersistencia.buscar(
            "Integrante",
            "nroLegajoIntegrante = " + legajoLider
        );

        if (integranteList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el líder");
        Integrante lider = integranteList.get(0);

        if (lider.getFechaBajaIntegrante() != null)
            throw new RuntimeException("Error: el líder está dado de baja");

        List<Object> proyectoList = IndireccionPersistencia.buscar(
            "Proyecto",
            "nroProyecto = " + nroProyecto
        );

        if (proyectoList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el proyecto");
        Proyecto proyecto = (Proyecto) proyectoList.get(0);

        if (!proyecto.getEstadoProyecto().getNombreEstadoProyecto().equals("PENDIENTE"))
            throw new RuntimeException("Error: el proyecto no se puede iniciar");

        Cliente cliente = proyecto.getCliente();

        if (cliente.getAlgoritmoPriorización().getNombre().equals("Equipos Cloud")) {
            // Lógica necesaria para obtener los equipos con más arquitectos Cloud
        } else if (cliente.getAlgoritmoPriorización().getNombre().equals("Experiencia previa")) {
            // Lógica necesaria para obtener a los equipos con más experiencia previa con el cliente
        } else {
            throw new IllegalArgumentException("Error: No es un algoritmo de priorización válido");
        }
        // Luego mostrar el equipo por pantalla
    }
}
```

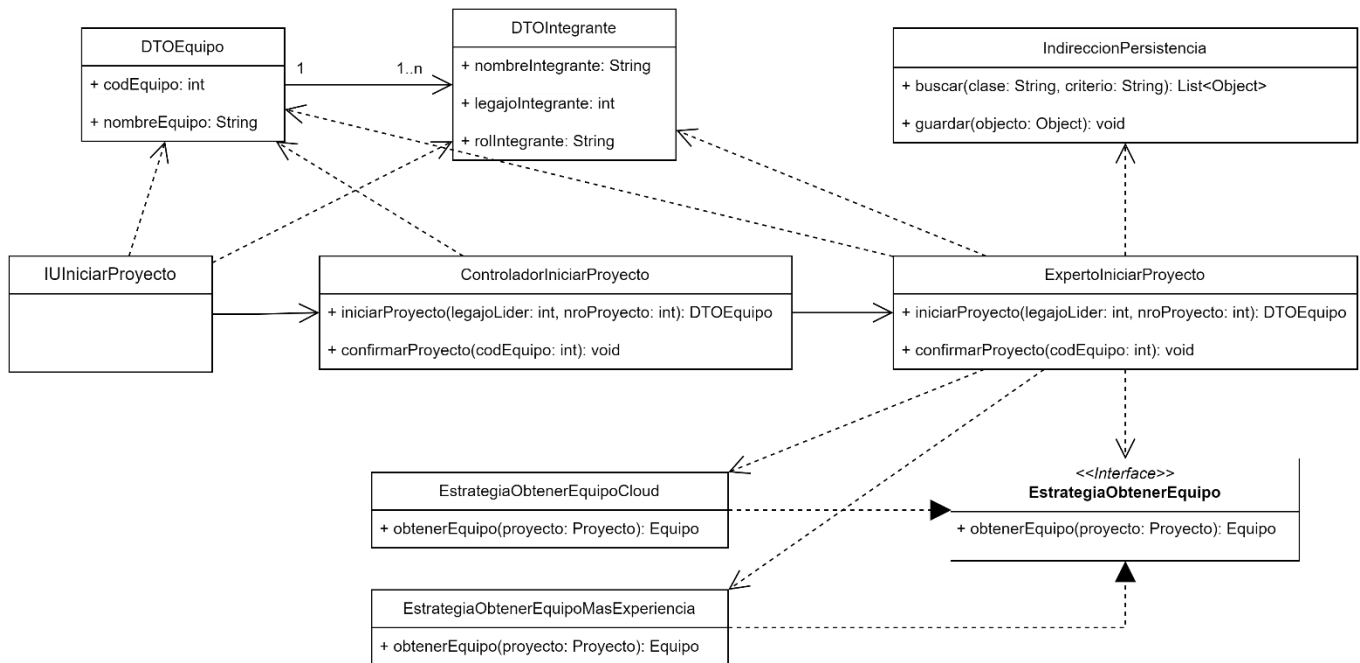
Sin embargo, se presentan algunos problemas:

- La legibilidad del código se dificulta, al tener un bloque condicional por cada algoritmo de selección de equipo.
- La adición, eliminación, o modificación de algún algoritmo implica que vamos a tener que modificar el código del experto, por lo que queda acoplado a los posibles algoritmos. Si además cada algoritmo requiere de otras clases para funcionar, estas dependencias recaen sobre el experto. Por lo que queda, a su vez, con un alto acoplamiento a las dependencias de cada algoritmo.
- Se reduce la cohesión del Experto, al tener que conocer qué algoritmo utilizar y sus detalles de implementación se desvirtúa su responsabilidad original: permitir que un líder tome un proyecto y se le asigne un equipo.

Antes de continuar, es importante recordar el patrón GRASP **Variaciones Protegidas**, que nos dice que debemos identificar los puntos de cambio en el código y establecer una *interfaz estable* alrededor de los mismos. El punto de variación del que hay que protegernos es la obtención del equipo, ya sabemos por los requisitos que nos han dado que es algo que a futuro va a cambiar.

Para solucionar esto podemos recurrir al patrón **Estrategia**:

- PROBLEMA: *¿Cómo diseñar diversos algoritmos o políticas que están relacionadas? ¿Cómo diseñar que estos algoritmos o políticas puedan cambiar?*
- SOLUCIÓN: *Defina cada algoritmo/política/estrategia en una clase independiente con una interfaz común.*



El patrón Estrategia es una implementación de Variaciones Protegidas que nos propone definir cada algoritmo en una clase aparte, haciendo que implementen una misma interfaz que actuará como un GRASP indirección. El Experto solo debe interactuar mediante dicha interfaz utilizando un mensaje **polimórfico** que será implementado por cada estrategia de forma distinta.

Cuando el patrón Variaciones Protegidas menciona la “interfaz estable” se refiere a un componente que no cambie, es decir que la manera de interactuar con él sea siempre la misma. La forma que tenemos de implementar esto en Java es con interfaces (clases abstractas puras) bien diseñadas y polimorfismo.

Para que dicha interfaz esté bien diseñada, se requiere que el método tenga una signatura lo suficientemente flexible y extensible para evitar modificaciones futuras (ya que sino no estamos protegiéndonos bien de las variaciones). Por lo tanto, los parámetros y el valor de retorno deben ser adecuados para las estrategias actuales del sistema y las que puedan incorporarse en el futuro.

En nuestro caso, enviar el proyecto como parámetro y retornar el equipo seleccionado es una solución robusta, ya que este enfoque se mantendrá relevante independientemente de las nuevas estrategias que se implementen.

Se puede implementar el patrón Estrategia de la siguiente manera:

```
public class ExpertoIniciarProyecto {

    public void iniciarProyecto(int legajoLider, int nroProyecto) throws Exception {

        List<Object> integranteList = IndireccionPersistencia.buscar(
            "Integrante",
            "nroLegajoIntegrante = " + legajoLider
        );

        if (integranteList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el líder");

        Integrante lider = integranteList.get(0);

        if (lider.getFechaBajaIntegrante() != null)
            throw new RuntimeException("Error: el líder está dado de baja");

        List<Object> proyectoList = IndireccionPersistencia.buscar(
            "Proyecto",
            "nroProyecto = " + nroProyecto
        );

        if (proyectoList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el proyecto");

        Proyecto proyecto = (Proyecto) proyectoList.get(0);

        if (!proyecto.getEstadoProyecto().getNombreEstadoProyecto().equals("PENDIENTE"))
            throw new RuntimeException("Error: el proyecto no se puede iniciar");

        Cliente cliente = proyecto.getCliente();

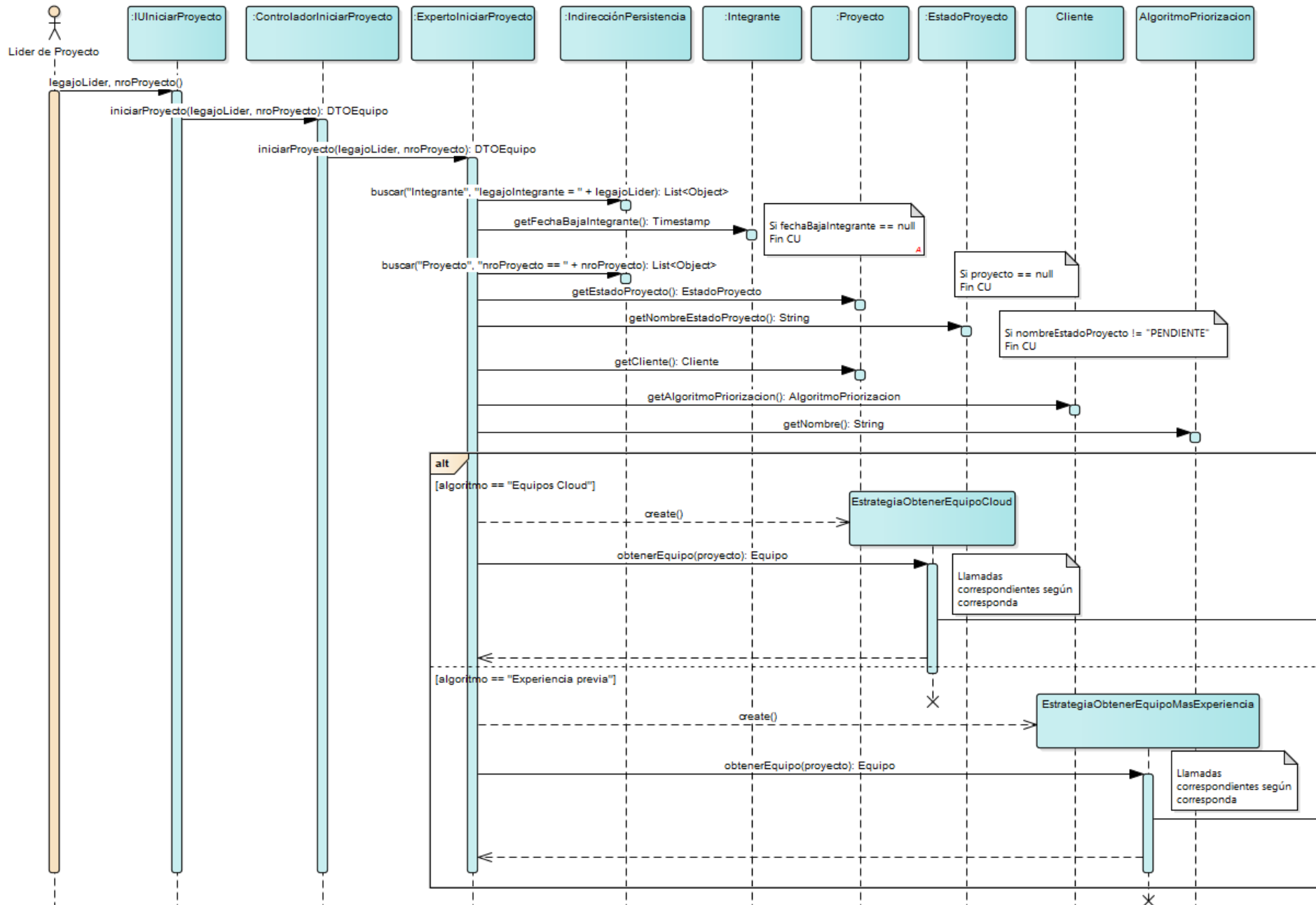
        EstrategiaObtenerEquipo estrategia = null;

        if (cliente.getAlgoritmoPriorización().getNombre().equals("Equipos Cloud")) {
            estrategia = new EstrategiaObtenerEquipoCloud();
        } else if (cliente.getAlgoritmoPriorización().getNombre().equals("Experiencia previa")) {
            estrategia = new EstrategiaObtenerEquipoMasExperiencia();
        } else {
            throw new IllegalArgumentException("Error: No es un algoritmo de priorización válido");
        }

        Equipo equipoSeleccionado = estrategia.obtenerEquipo(proyecto); //POLIMORFISMO
        // Luego mostrar el equipo por pantalla
    }
}
```

En un diagrama de secuencia, debido a las limitaciones de UML, no es posible modelar directamente el polimorfismo. Esto se debe a que este tipo de diagramas representan la interacción entre objetos, y una interfaz no puede ser instanciada. Aunque el uso del patrón implica polimorfismo, es necesario mostrar el envío de mensajes a cada una de las estrategias concretas, detallando cómo implementan los métodos definidos en la interfaz. Por lo tanto, es incorrecto modelar un único mensaje dirigido directamente hacia la interfaz.

Con lo que sabemos hasta el momento, así debería verse un diagrama de secuencia utilizando el patrón estrategia. Queda como trabajo para el lector modelar en el diagrama de secuencia la implementación de cada una de las estrategias.



¿Se resolvió el problema? La respuesta es que no. Esto es una solución incompleta al problema presentado: Se ve en el código que el experto todavía debe conocer qué estrategia instanciar, por lo que su cohesión sigue siendo baja. Además, al tener que instanciar a la estrategia que corresponde se genera un acoplamiento con las mismas.

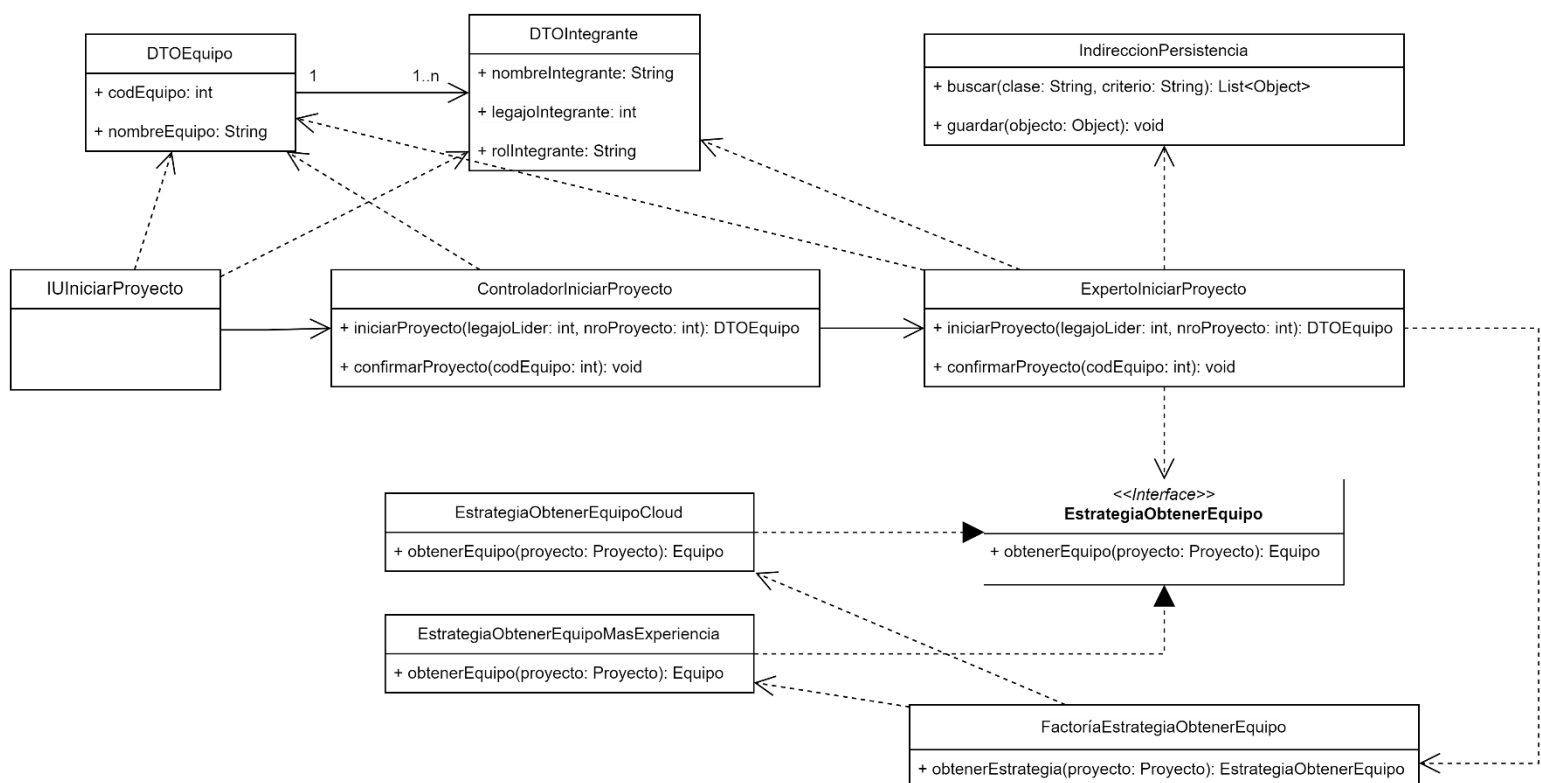
Esto es porque el patrón Estrategia divide el problema que teníamos en dos partes:

- La implementación de los algoritmos en sí mismos, mediante el patrón estrategia
- La forma de definir qué algoritmo utilizar, que para resolverlo debemos recurrir a otro patrón

Factoría

Al utilizar el patrón anterior, si bien nuestro código mejoró, sigue existiendo cierto grado acoplamiento entre el Experto y las estrategias concretas. Esto es porque el Experto todavía tiene la responsabilidad de instanciar a la estrategia requerida, lo cual además no tiene relación alguna con su funcionalidad principal. Entonces podemos utilizar un patrón creacional llamado **Factoría**:

- PROBLEMA: ¿Quién debe ser el responsable de la creación de objetos cuando existen consideraciones especiales, como una lógica de creación compleja, el deseo de separar las responsabilidades de la creación para mejorar la cohesión, etc.?
- SOLUCIÓN: Crear un objeto **Fabricación Pura** llamado Factoría que se encargue de gestionar la creación de objetos.



Como no queremos que el Experto conozca los criterios para decidir qué estrategia utilizar, buscamos trasladar dicha responsabilidad a otra clase. Como indica el GRASP **Fabricación Pura**, si no existe ninguna clase a la cual asignarle una responsabilidad que mantenga la cohesión alta y el acoplamiento bajo, entonces debemos “inventar” una. En este caso será una clase denominada “Factoría” que ofrece servicios a otras clases (como el Experto de nuestro caso de uso), encargándose de decidir qué crear y retornarlo, de forma tal que el resto de clases no conozca la lógica que hay detrás.

```
public class FactoriaEstrategiaObtenerEquipo {

    private static Map<String, EstrategiaObtenerEquipo> mapEstrategias = new HashMap<>();

    static {
        mapEstrategias.put("Equipos Cloud", new EstrategiaObtenerEquipoCloud());
        mapEstrategias.put("Experiencia previa", new EstrategiaObtenerEquipoMasExperiencia());
    }

    public EstrategiaObtenerEquipo obtenerEstrategia(Proyecto proyecto) {
        Cliente cliente = proyecto.getCliente();

        EstrategiaObtenerEquipo estrategia = mapEstrategias.get(cliente.getAlgoritmoPriorización().getNombre());

        if (estrategia == null) throw new IllegalArgumentException("Error: No se encontró un algoritmo válido");

        return estrategia;
    }
}

public class ExpertoIniciarProyecto {

    public void iniciarProyecto(int legajoLider, int nroProyecto) throws Exception {
        List<Object> integranteList = IndireccionPersistencia.getInstance().buscar(
            "Integrante",
            "nroLegajoIntegrante = " + legajoLider
        );

        if (integranteList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el líder");
        Integrante lider = integranteList.get(0);

        if (lider.getFechaBajaIntegrante() != null)
            throw new RuntimeException("Error: el líder está dado de baja");

        List<Object> proyectoList = IndireccionPersistencia.getInstance().buscar(
            "Proyecto",
            "nroProyecto = " + nroProyecto
        );

        if (proyectoList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el proyecto");

        Proyecto proyecto = (Proyecto) proyectoList.get(0);

        if (!proyecto.getEstadoProyecto().getNombreEstadoProyecto().isEquals("PENDIENTE"))
            throw new RuntimeException("Error: el proyecto no se puede iniciar");

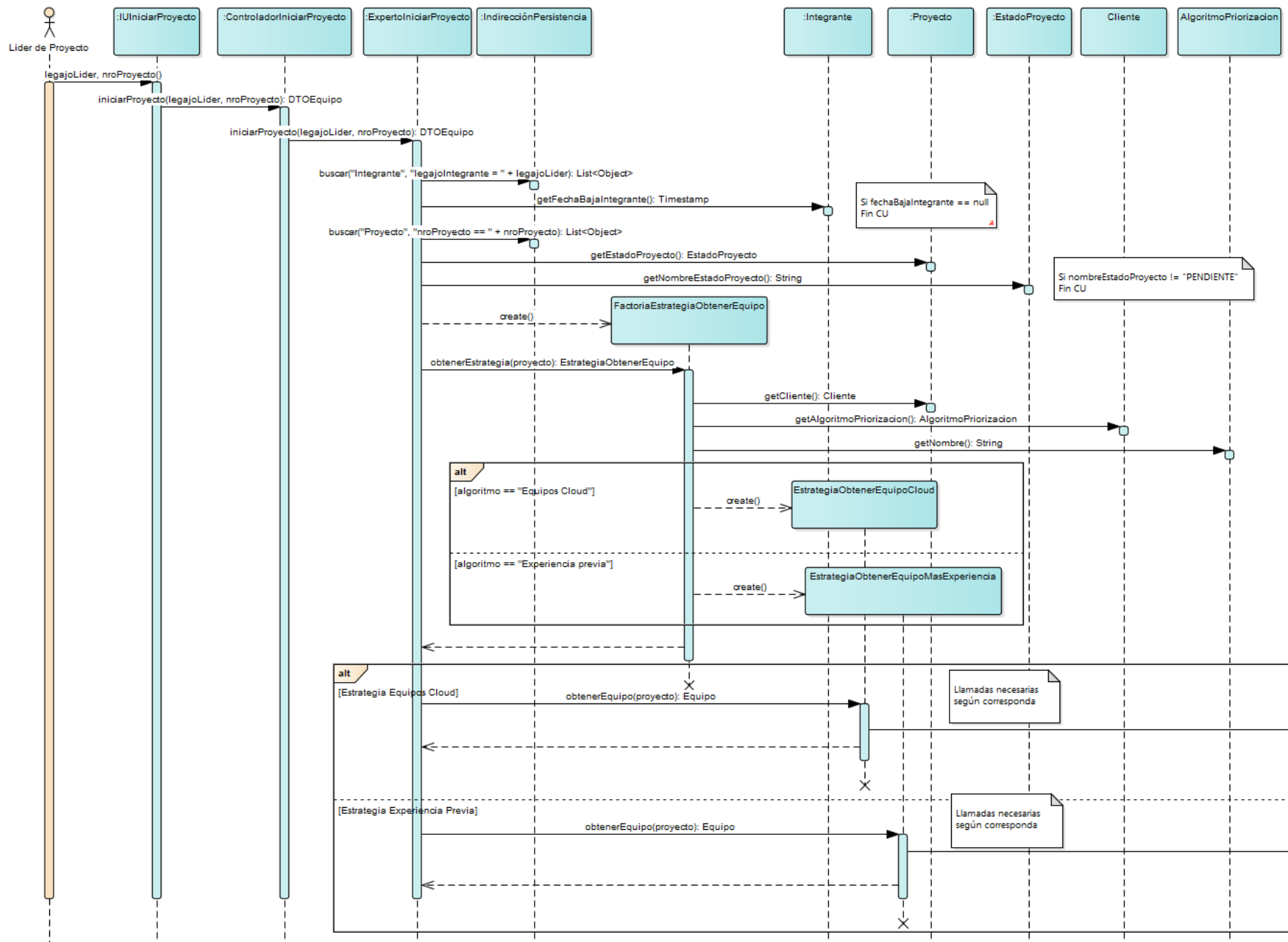
        FactoriaEstrategiaObtenerEquipo factoria = new FactoriaEstrategiaObtenerEquipo();

        EstrategiaObtenerEquipo estrategia = factoria.obtenerEstrategia(proyecto);

        Equipo equipoRecomendado = estrategia.obtenerEquipo(proyecto); //POLIMORFISMO

        // Luego mostrar el equipo por pantalla
    }
}
```

Es importante definir qué parámetro pasarle a la factoría para que pueda determinar qué es lo que tiene que instanciar. Debemos enviarle aquello que no pueda determinar por sí misma: en este caso no puede determinar el proyecto ni el cliente ya que debe ser el que veníamos utilizando anteriormente en el Experto, entonces debemos pasárselo como parámetro para que pueda decidir qué estrategia retornar. Puede ocurrir que haya algún dato que requiera y que lo determine por sí misma, utilizando alguna operación de lectura a la base de datos, leyendo algún archivo de configuración, conectándose a una API, etc.



Con esto la separación de responsabilidades mejora considerablemente. Sin embargo, se genera un problema adicional, ¿quién debe instanciar la factoría?

Singleton

Sabemos que no tenemos a nadie a quién asignarle la responsabilidad de crear la Factoría, y sabemos que sólo necesitaremos una única instancia de la misma, que puede ser requerida en distintos puntos de nuestro sistema. Entonces podemos utilizar el patrón de diseño **Singleton**:

- PROBLEMA: *Se admite exactamente una única instancia de una clase (por eso "Singleton") que debe ser accedida desde un punto de acceso global.*
- SOLUCIÓN: *Defina un método estático de la clase que devuelva el Singleton*

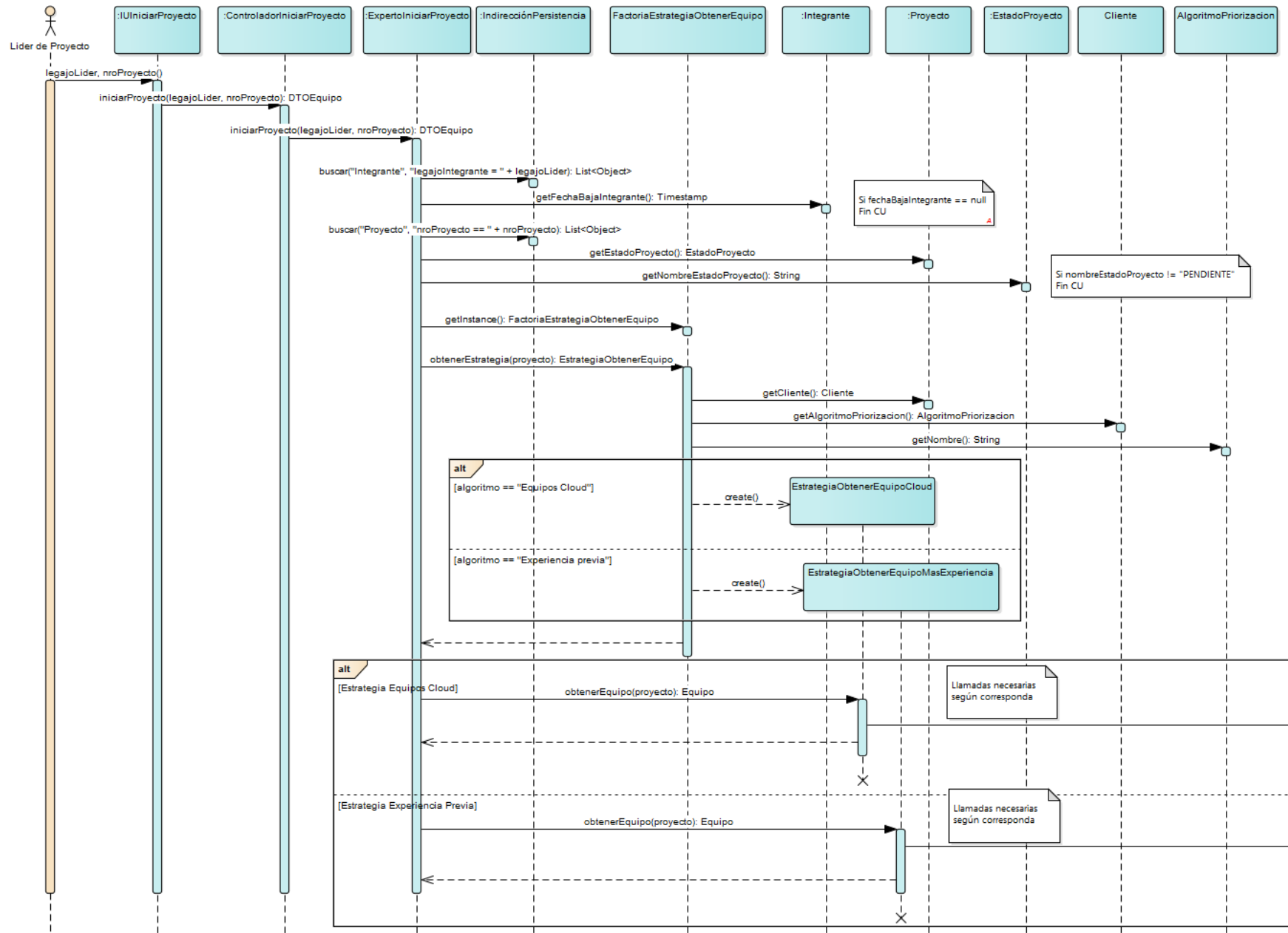
```
public class FactoriaEstrategiaObtenerEquipo {  
  
    private static FactoriaEstrategiaObtenerEquipo instance;  
  
    private static Map<String, EstrategiaObtenerEquipo> mapEstrategias = new HashMap<>();  
  
    static {  
        mapEstrategias.put("Equipos Cloud", new EstrategiaObtenerEquipoCloud());  
        mapEstrategias.put("Experiencia previa", new EstrategiaObtenerEquipoMasExperiencia());  
    }  
  
    private FactoriaEstrategiaObtenerEquipo() {}; //El constructor es privado para que nadie pueda acceder  
  
    public static FactoriaEstrategiaObtenerEquipo getInstance() {  
  
        if (FactoriaEstrategiaObtenerEquipo.instance == null) {  
            FactoriaEstrategiaObtenerEquipo.instance = new FactoriaEstrategiaObtenerEquipo();  
        }  
  
        return FactoriaEstrategiaObtenerEquipo.instance;  
    }  
  
    public EstrategiaObtenerEquipo obtenerEstrategia(Proyecto proyecto) {  
  
        Cliente cliente = proyecto.getCliente();  
  
        EstrategiaObtenerEquipo estrategia = mapEstrategias.get(cliente.getAlgoritmoPriorización().getNombre());  
  
        if (estrategia == null) throw new IllegalArgumentException("Error: No se encontró un algoritmo válido");  
  
        return estrategia;  
    }  
}
```

El constructor de la Factoría se declara como privado para evitar que otras clases puedan instanciarla directamente. La única forma de obtener una instancia de este Singleton es a través del método estático `getInstance`. Este método verifica si la instancia ya existe; si no, la crea y la asigna al atributo estático correspondiente. De esta manera, se garantiza que siempre se retorne la misma instancia, permitiendo un acceso controlado y consistente desde cualquier parte del código. Además, dentro del constructor se puede añadir lógica adicional en caso de ser necesario. La instanciación del Singleton se puede diferir hasta que sea requerido (*lazy initialization*) o se puede instanciar de forma anticipada (*eager initialization*), según cómo se implemente el método `getInstance`.

Ya que este patrón permite que un objeto sea de alcance global, hay que tener en consideración que debe ser una clase que no tenga estado (es decir, que no posea atributos además del Singleton). En la mayoría de casos, no se recomienda utilizar variables globales, ya que como podrían ser modificada en cualquier parte del código, los valores de sus atributos en un momento dado pueden ser inconsistentes o distintos de lo esperado, lo que puede provocar efectos secundarios y errores en tiempo de ejecución.

¿Hay alguna otra clase de la que se requiera una única instancia en nuestro diseño? La respuesta es sí, la indirección de persistencia ya que es una fachada a un sistema de persistencia, por lo que podemos aplicar también el patrón Singleton con ella. Hasta ahora, habíamos ignorado cómo hacía el experto para obtener una referencia a la indirección.

Entonces la solución final para el problema nos quedaría de la siguiente manera:



```

public class ExpertoIniciarProyecto {

    public void iniciarProyecto(int legajoLider, int nroProyecto) throws Exception {
        List<Object> integranteList = IndireccionPersistencia.getInstance().buscar(
            "Integrante",
            "nroLegajoIntegrante = " + legajoLider
        );

        if (integranteList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el líder");
        Integrante lider = integranteList.get(0);

        if (lider.getFechaBajaIntegrante() != null)
            throw new RuntimeException("Error: el líder está dado de baja");

        List<Object> proyectoList = IndireccionPersistencia.getInstance().buscar(
            "Proyecto",
            "nroProyecto = " + nroProyecto
        );

        if (proyectoList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el proyecto");

        Proyecto proyecto = (Proyecto) proyectoList.get(0);

        if (!proyecto.getEstadoProyecto().getNombreEstadoProyecto().isEquals("PENDIENTE"))
            throw new RuntimeException("Error: el proyecto no se puede iniciar");

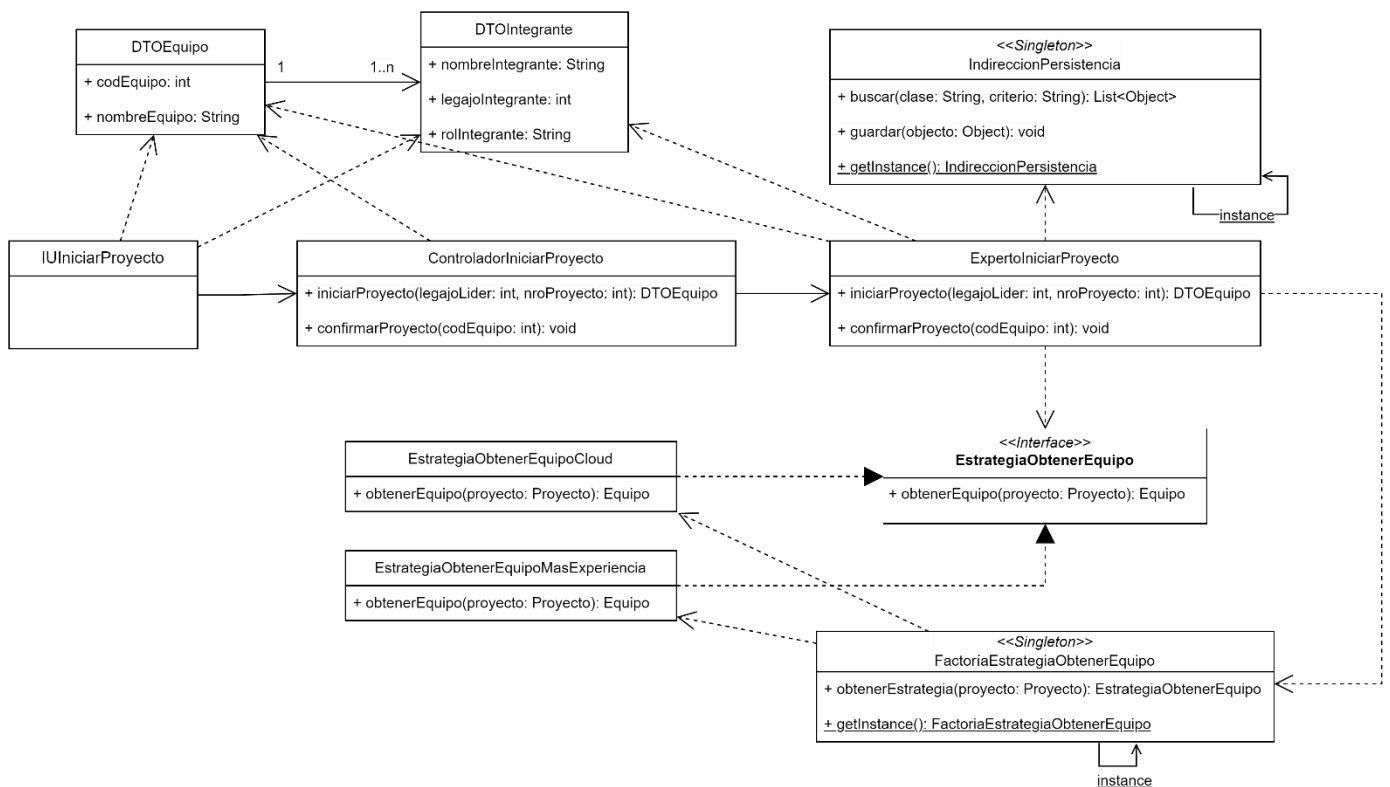
        FactoriaEstrategiaObtenerEquipo factoria = FactoriaEstrategiaObtenerEquipo.getInstance();

        EstrategiaObtenerEquipo estrategia = factoria.obtenerEstrategia(proyecto);

        Equipo equipoRecomendado = estrategia.obtenerEquipo(proyecto); //POLIMORFISMO

        // Luego mostrar el equipo por pantalla
    }
}

```



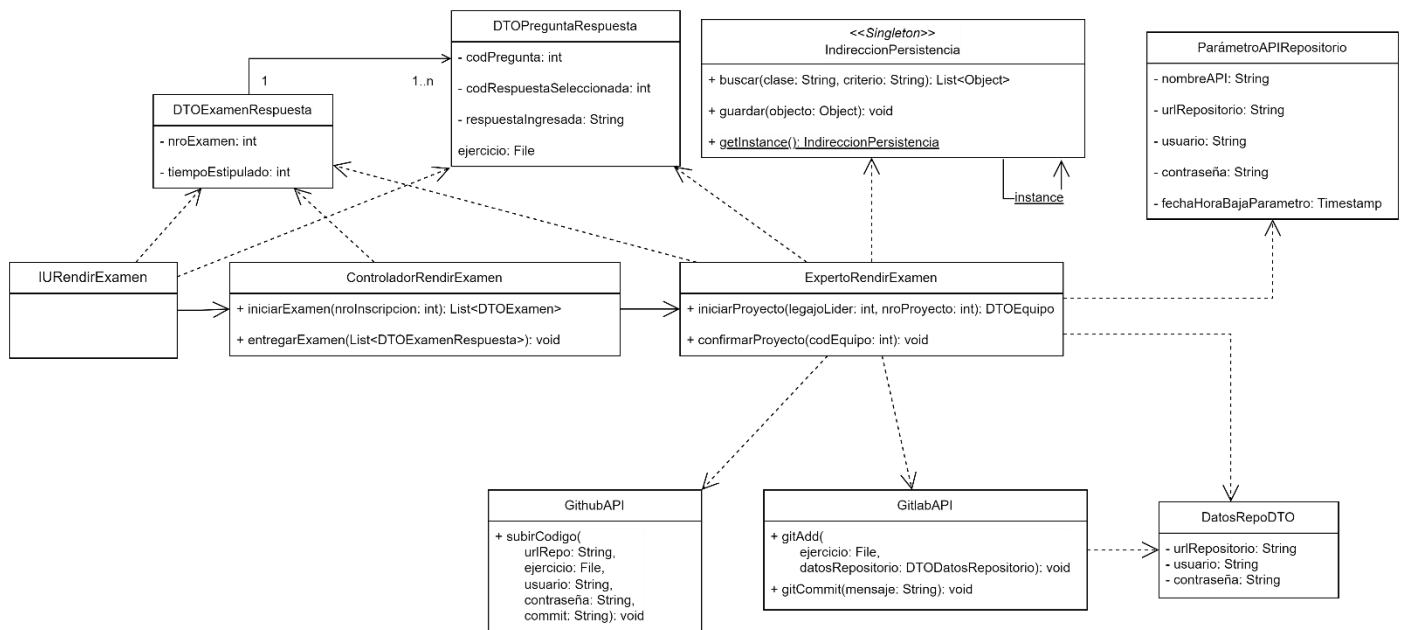
Se ve que ahora el experto tiene total desconocimiento de qué estrategia está utilizando, ya que no tiene que instanciarla y además usa un mensaje polimórfico para interactuar con la interfaz. Entonces ahora sí nos queda desacoplado de los algoritmos, y con una cohesión alta. Si bien tuvimos que añadir más clases, el código del experto se ve mucho más sencillo que al inicio, es mucho más mantenible y escalable, ya que si tuviésemos que realizar algún cambio en los algoritmos (como añadir, eliminar, o modificar una estrategia) no es necesario alterar el código del experto. Simplemente tendríamos que crear una nueva clase Estrategia que implemente la interfaz y modificar la Factoría.

Adaptador

Los municipios realizan programas en los que se evalúa a los candidatos inscriptos. En cada programa se toma una serie de exámenes de donde se selecciona a aquellos que cumplan con una nota mínima. Al final de todos los exámenes el candidato debe resolver un ejercicio de programación, para lo cual debe adjuntar un único archivo con el código fuente desarrollado.

Se requiere poder almacenar los distintos entregables de cada candidato en un repositorio central. Actualmente la empresa se maneja con Github pero planean a futuro migrar a Gitlab o a Bitbucket. La API a utilizar es determinada como una configuración del sistema únicamente modificable por el administrador del mismo.

Esto presenta una limitación: cada servicio de terceros expone una interfaz distinta y no puede ser modificada por nosotros. Entre las distintas APIs se utilizan métodos distintos y parámetros distintos. Se nos pide realizar el caso de uso “Rendir Examen”, únicamente la lógica relacionada a entregar el código. Podríamos imaginar el siguiente ejemplo:



```
public class ExpertoRenderExamen {

    public void entregarExamen(List<DTOExamenRespuesta> examenList, File ejercicio) throws Exception{
        // ...
        List<Object> parametroApiList = IndireccionPersistencia.getInstance().buscar(
            "ParametroAPIRepositorio",
            "fechaHoraBajaParametro == null");
        if (parametroApiList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el parámetro a utilizar");

        ParametroAPIRepositorio parametroApi = (ParametroAPIRepositorio) parametroApiList.get(0);

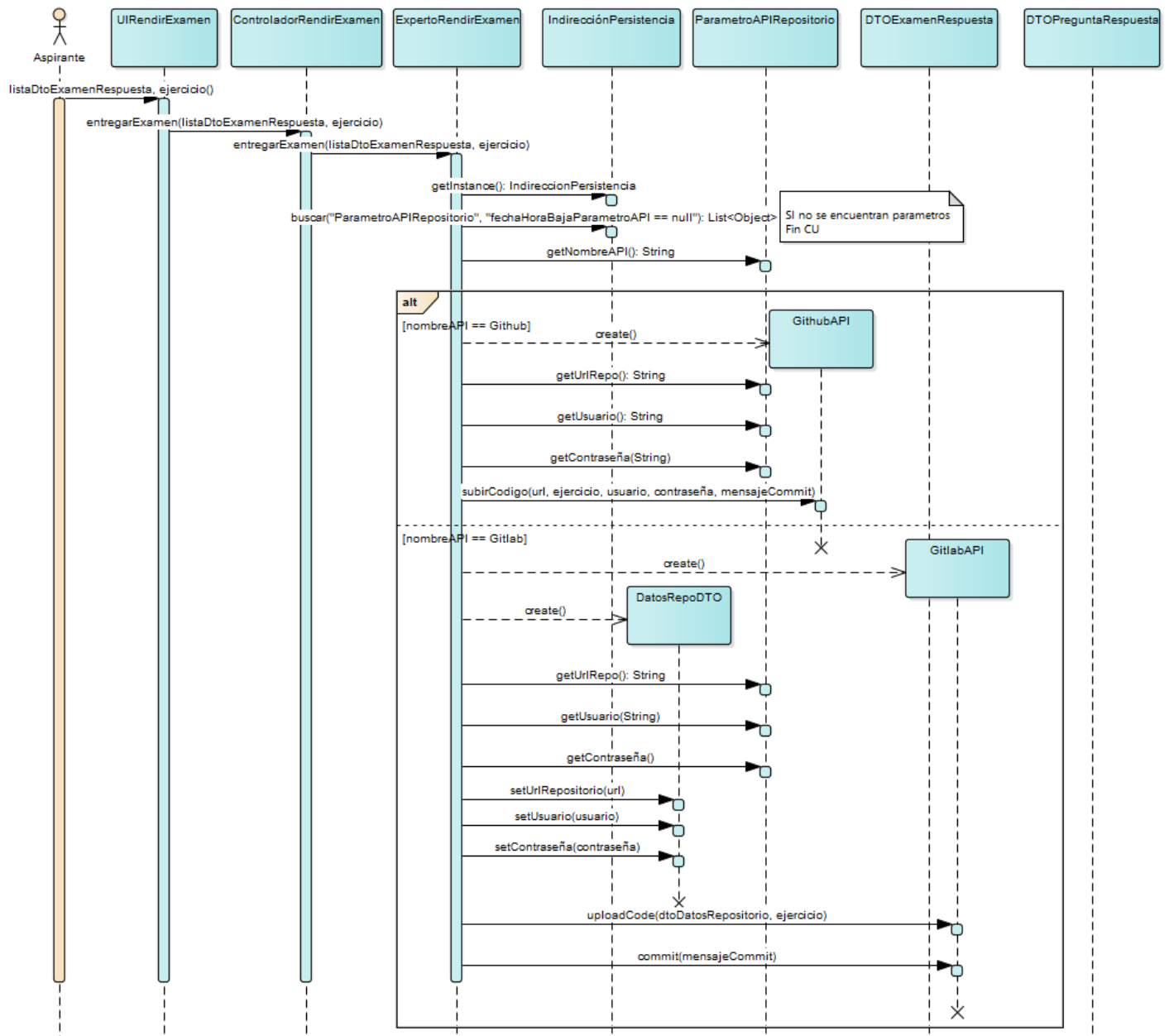
        if (parametroApi.getNombreAPI().equals("Github")) {
            GithubAPI github = new GithubAPI();

            github.subirCodigo(
                parametroApi.getUrlRepo(),
                ejercicio,
                parametroApi.getUsuario(),
                parametroApi.getContraseña(),
                "Mensaje del commit");
        } else if (parametroApi.getNombreAPI().equals("Gitlab")) {
            GitlabAPI gitlab = new GitlabAPI();

            DatosRepoDTO dtoDatosRepositorio = new DTODatosRepositorio(
                parametroApi.getUrlRepo(),
                parametroApi.getContraseña(),
                parametroApi.getUsuario());

            gitlab.uploadCode(
                dtoDatosRepositorio,
                ejercicio);

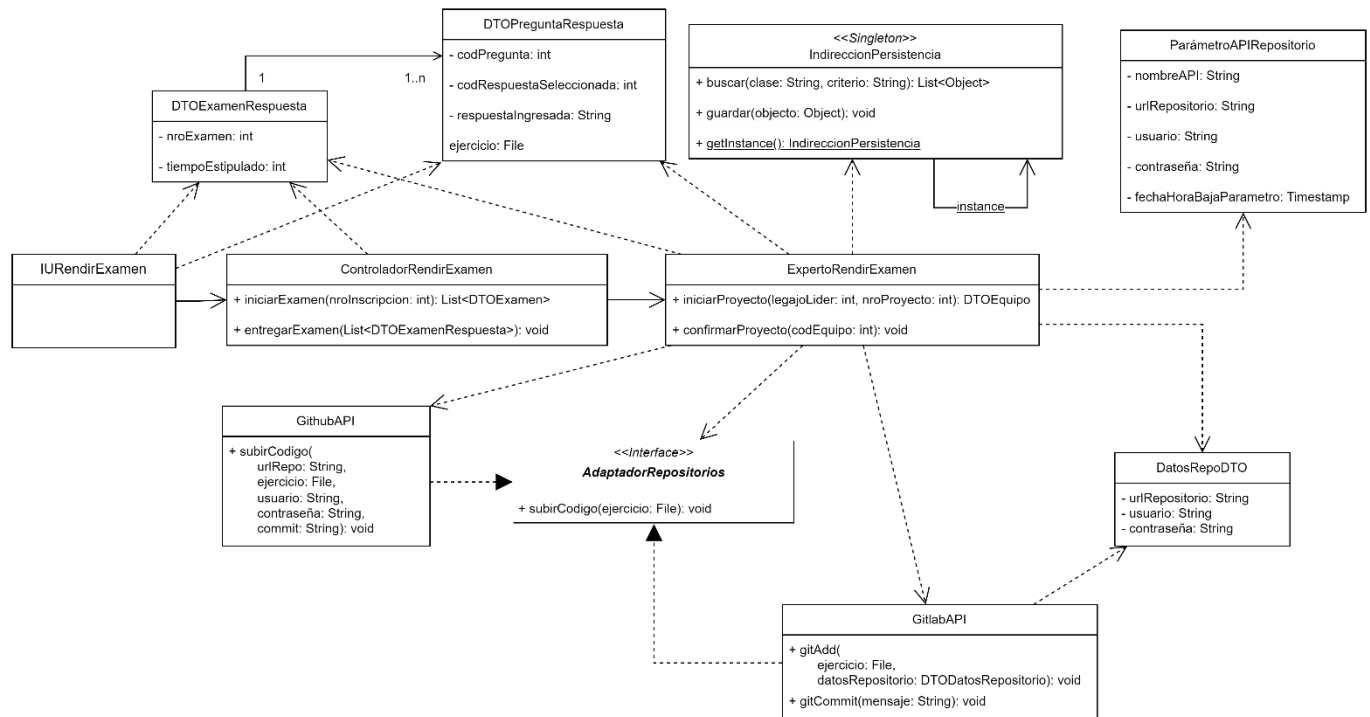
            gitlab.commit("Mensaje del commit");
        } else {
            throw new IllegalArgumentException("Error: No se encontró la API a utilizar");
        }
        // ...
    }
}
```



Nuevamente se encuentran algunos problemas en esta solución:

- Se dificulta la legibilidad del código al tener un bloque condicional por cada posible API a utilizar.
- La eliminación, adición, o modificación (por parte de terceros) de una API al sistema implica modificar el Experto. Esto es porque dicha clase está acoplada con cada uno de los servicios externos.
- El Experto tiene que tener conocimiento de cómo interactuar con cada una de las APIs y decidir cuál utilizar. Por lo tanto, debe conocer exactamente qué parámetros recibe cada una, sus métodos, sus valores de retorno, etc. Esto hace que se desvirtúe su responsabilidad principal, que es la de permitir que un aspirante rinda un examen. Entonces podemos afirmar que su cohesión no es tan alta como podría ser.

Podemos pensar nuevamente en el patrón **Variaciones Protegidas**: tenemos identificado el punto de variación, que es la lógica necesaria para subir el archivo al repositorio central, así que utilizamos una interfaz para protegernos de los cambios futuros en el código. Dicha interfaz actúa como indirección entre las APIs a utilizar. Recordemos que siempre que utilicemos el patrón Variaciones Protegidas la idea es diseñar la interfaz para que sea lo más flexible posible, es decir, que los métodos que definamos nos protejan de las variaciones actuales y futuras. Tener un buen diseño de interfaces nos permite realizar abstracciones robustas de los componentes de nuestro sistema, que a la larga hacen que sea más mantenible.

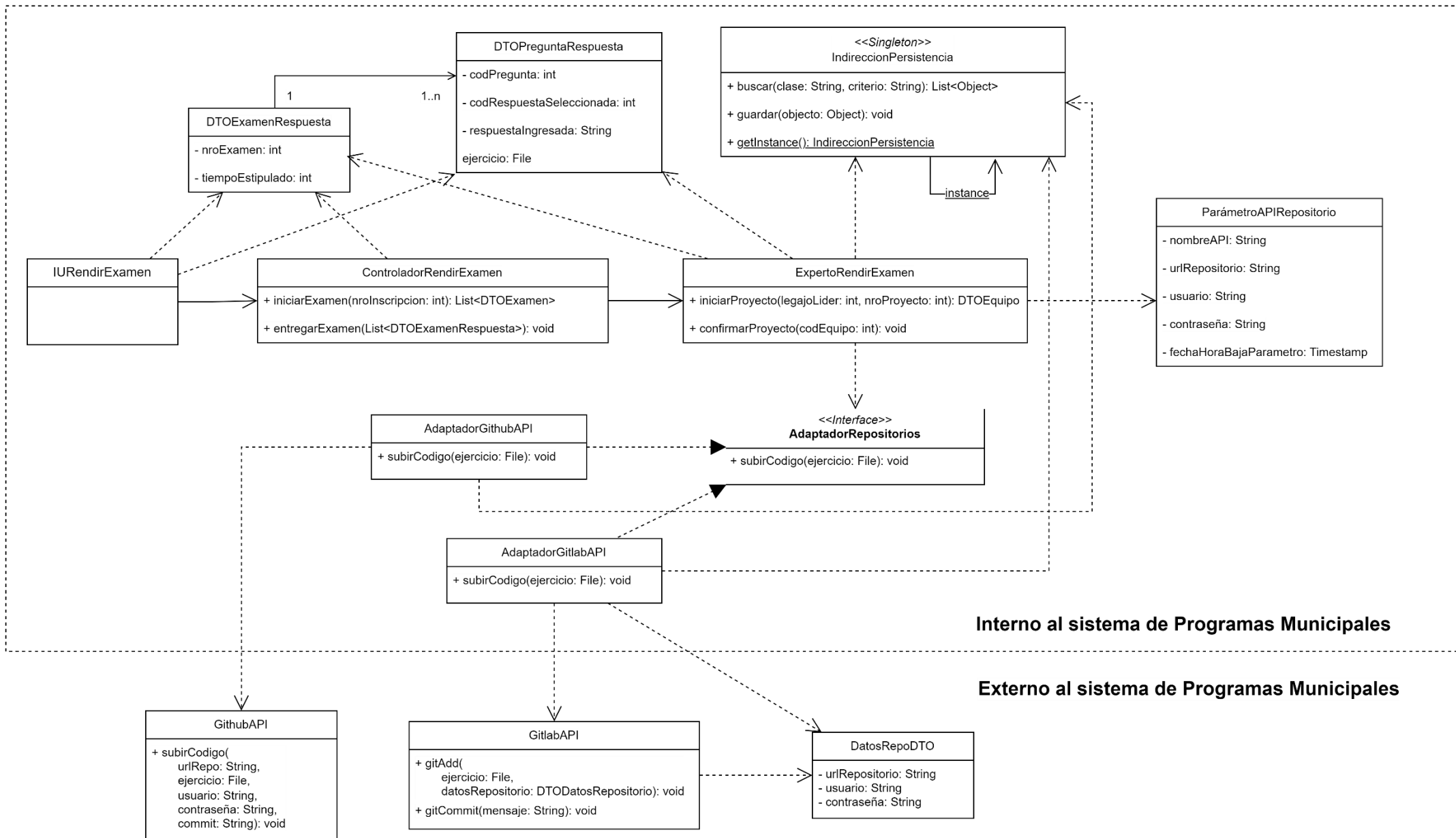


Sin embargo, en la imagen de arriba se ve que nuestra indirección tiene un método llamado “subirCodigo()”. La API de Github tiene un método llamado igual y que recibe 5 parámetros distintos. En cambio, la API de Gitlab tiene 2 métodos con otro nombre de los cuales uno de ellos recibe un DTO, en conjunto esos dos métodos realizan la misma funcionalidad que Github resuelve con una sola llamada.

Al inicio de la sección mencionamos que como las APIs son proporcionadas por terceros, no podemos modificarlas. Cada empresa diseñó su interfaz como le parecía conveniente. Por lo tanto, tienen nombres distintos para los métodos, reciben parámetros distintos, tienen tipos de retorno distintos, etc. Es decir, no podemos forzarlos a implementar nuestra interfaz, son incompatibles. Esto implica que no podemos utilizar polimorfismo, ya que no se puede utilizar el mismo mensaje para todas las APIs, lo que significa que la interfaz que utilizamos para tener variaciones protegidas no puede cumplir su propósito.

Con todo lo mencionado, ahora sí podemos introducir el patrón **Adaptador**:

- **PROBLEMA:** *¿Cómo resolver interfaces incompatibles, o proporcionar una interfaz estable para componentes parecidos con diferentes interfaces?*
- **SOLUCIÓN:** *Convierta la interfaz original de un componente en otra interfaz mediante un objeto adaptador intermedio.*



Nos dice que debemos introducir una clase que “adapte” nuestra interfaz a la proporcionada por cada uno de los servicios externos.

Cada adaptador concreto debe implementar la interfaz que definimos como indirección, por lo tanto están forzados a implementar el método “subirCodigo()”. Dicho método debe adaptar una única llamada polimórfica a todas las necesarias para interactuar con la API que corresponde. Por ejemplo, el adaptador de Gitlab debe incluir en su método las llamadas a los dos métodos necesarios según la API de Gitlab y encargarse brindarle los parámetros necesarios. El adaptador de Github, por su parte, debe hacer lo mismo para el caso de la API de Github.

El código mejoró, ahora el Experto no conoce cómo interactuar con cada API, sino que esa responsabilidad fue trasladada al adaptador. Esto nos permite tener **Variaciones Protegidas** al añadir, eliminar, o modificar alguna de las APIs el Experto no sufrirá cambios sino el adaptador correspondiente, quedando desacoplado de las APIs.

Sin embargo, no hemos terminado de resolver todos los problemas. Se invita al lector a intentar pensar cuál es el problema pendiente y una posible solución antes de avanzar con la lectura.

Análogamente a como se vio en el patrón Estrategia, nuevamente nuestro problema se divide en dos: la interacción con los servicios externos y la instanciación del adaptador a utilizar. Tal y como está el código, el Experto debe encargarse de instanciar el adaptador que necesita. Como debe conocer qué adaptador crear, su cohesión disminuye y queda acoplado a los mismos. Tenemos un problema bastante similar al de la estrategia, por lo que el patrón Factoría y Singleton nos pueden ayudar nuevamente.

```
public class FactoriaAdaptadorRepositorios {

    private static Map<String, AdaptadorRepositorios> mapAdaptador = new HashMap<>();

    private static FactoriaAdaptadorRepositorios instance;

    static {
        mapAdaptador.put("Github", new AdaptadorGithubAPI());
        mapAdaptador.put("Gitlab", new AdaptadorGitlabAPI());
    }

    public AdaptadorRepositorios obtenerAdaptador() {

        List<Object> parametroApiList = IndireccionPersistencia.getInstance().buscar(
            "ParametroAPIRepositorio",
            "fechaBajaParametro == null");

        if (parametroApiList.isEmpty()) throw new EntityNotFoundException("Error: no se encontró el parámetro a utilizar");

        ParametroAPIRepositorio parametroApi = (ParametroAPIRepositorio) parametroApiList.get(0);

        AdaptadorRepositorios adaptador = mapAdaptador.get(parametroApi.getNombreAPI());

        if (adaptador == null) throw new IllegalArgumentException("Error: No se encontró el adaptador a utilizar");

        return adaptador;
    }

    public static FactoriaAdaptadorRepositorios getInstance() {
        if (Factoria.instance == null) {
            Factoria.instance = new FactoriaAdaptadorRepositorios();
        }
        return Factoria.instance;
    }

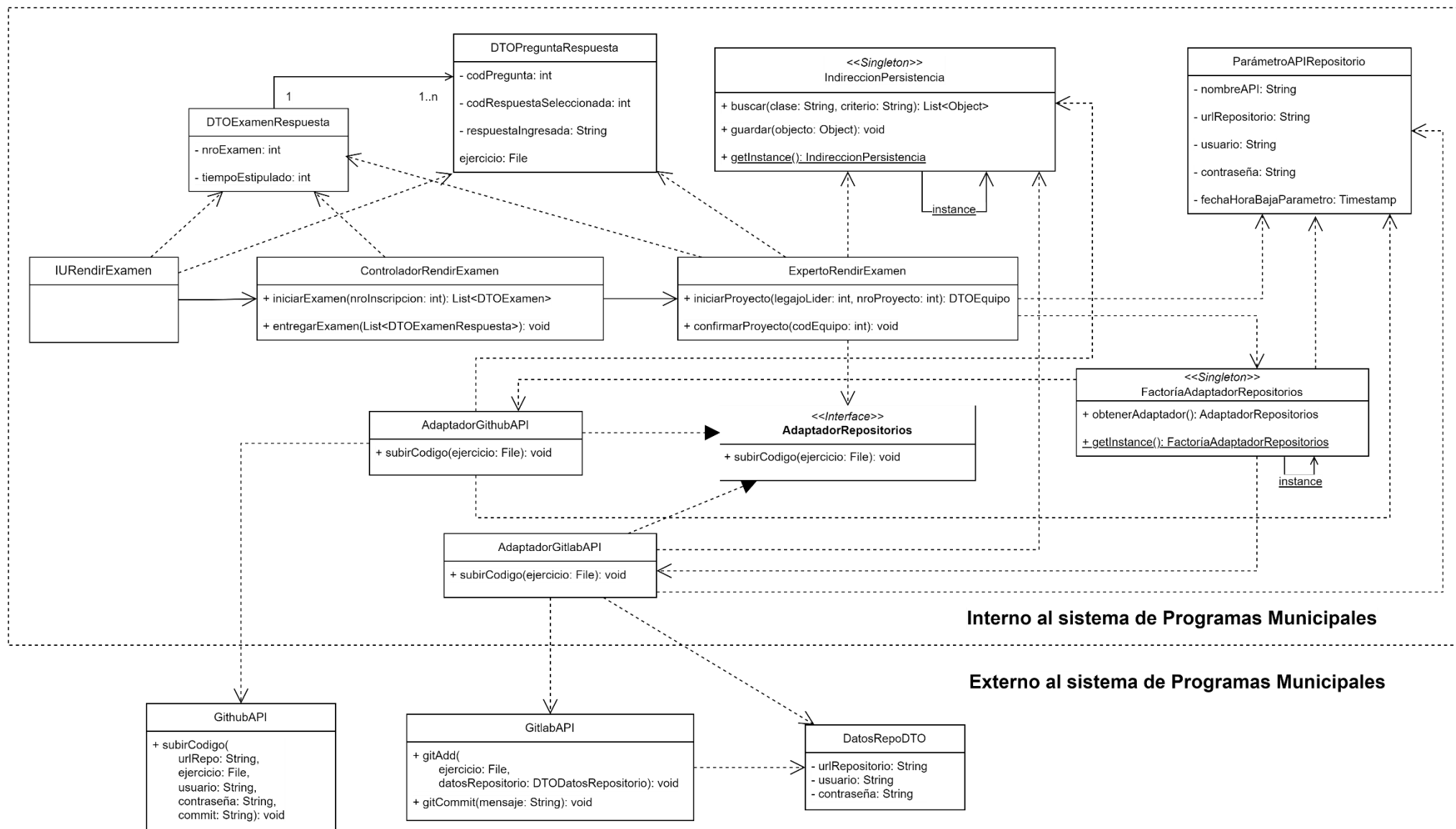
    private FactoriaAdaptadorRepositorios() {}
}

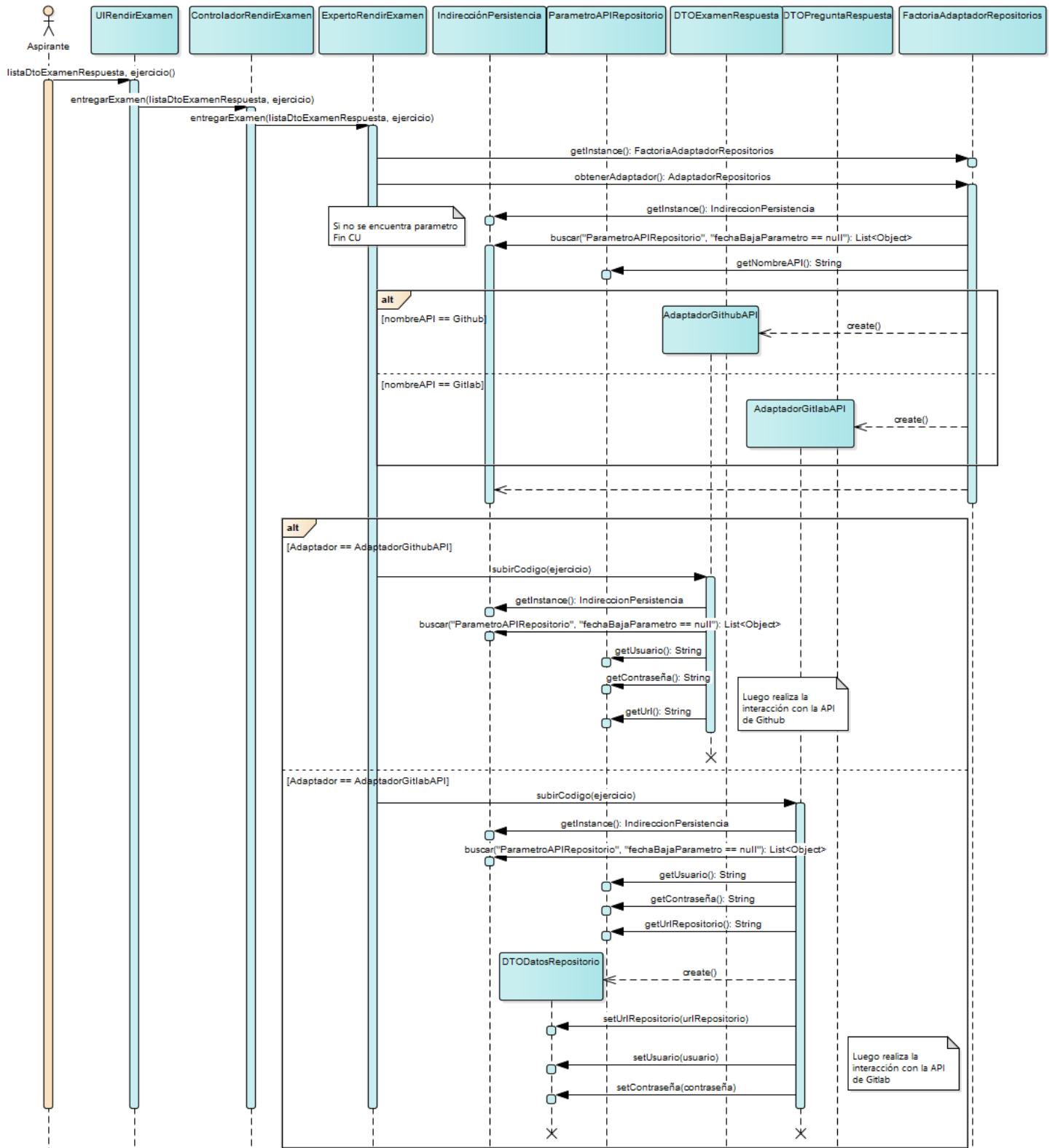
public class ExpertoRendirExamen {

    public void entregarExamen(List<DTOExamenRespuesta> examenList, File ejercicio) {
        // ...
        FactoriaAdaptadorRepositorios factoria = FactoriaAdaptadorRepositorios.getInstance();

        AdaptadorRepositorios adaptador = factoria.obtenerAdaptador();

        adaptador.subirCodigo(ejercicio);
        // ...
    }
}
```





En este caso, como los parámetros que necesita cada adaptador se obtienen de una tabla de la base de datos, no es necesario pasarles como parámetro nada más que el archivo con el ejercicio del candidato, el resto de datos lo puede obtener cada adaptador por su cuenta utilizando la Indirección de Persistencia.

Aclaración: No es una buena práctica almacenar contraseñas en una base de datos sin antes pasarlas por una función hash. Almacenar la contraseña en la tabla ParametroAPIRepositorio es solo por fines didácticos.

Ahora el experto no tiene conocimientos de qué adaptador utilizar, qué API utilizar, ni como interactuar con ellas: La interacción fue delegada a los adaptadores y la instanciación a la Factoría, que es accedida mediante un Singleton.

Conclusión

Logramos identificar problemas en nuestro diseño relacionados con la escalabilidad, mantenimiento, acoplamiento, y cohesión del mismo. Para solucionarlos, nos apoyamos en los patrones GRASP. Dichos patrones no ofrecen una única solución concreta a los problemas, sino que presentan principios generales que luego son implementados por otros patrones como lo son los GoF. Una vez entendidos los problemas y las soluciones que proponen los patrones, implementamos las mismas tanto en código como en diagramas UML de clases y de secuencia.

Es importante entender el propósito que tienen los patrones para saber adecuarlos a las necesidades que tengamos y saber discernir como diseñadores cuándo vale la pena aplicarlos. En palabras de Craig Larman “Elije tus batallas”.

Bibliografía

[1] Craig Larman. (2003). UML y Patrones 2da Edición, capítulos 16, 22, y 23. Editorial Pearson.

[2] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. (1994). Patrones de Diseño: Elementos de software orientado a objetos reusable. Editorial Addison Wesley.